Predicting Credit Defaults

Objective

To build a model to predict future credit defaulters by learning from past data.

Objective 1
Data 1
Features 1
Pre-processing 2
Inspection and cleaning 2
Categorical features 3
Correlation and analysis 4
Train Test Split 5
Model building 5
Logistic Regression 5
LDA 6
QDA 6
KNN 6
Gradient Boost 7
Random Forest 7
Catboost 7
Conclusion 7

Data

A dataset containing 1000 records of customers who have availed loans from a German bank has been provided.

Features

- checking_balance - Amount of money available in account of customers (range categories)
- months_loan_duration - Duration since loan taken
- credit_history - credit history of each customers (categorical info)
- purpose - Purpose why loan has been taken (categorical info)
- amount - Amount of loan taken

- savings_balance - Balance in account (range categories)
- employment_duration - Duration of employment
- percent_of_income - Percentage of monthly income
- years_at_residence - Duration of current residence
- age - Age of customer
- other_credit - Any other credits taken (categorical info)
- Housing - Type of housing, rent or own (categorical info)
- existing_loans_count - Existing count of loans
- job - Job type (categorical info)
- dependents - Any dependents on customer
- phone - Having phone or not (boolean)
- default - Default status (Target column)

Columns without an explicit type are numeric by default.

Pre-processing

Inspection and cleaning

```
#   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   checking_balance      1000 non-null   object
 1   months_loan_duration  1000 non-null   int64
 2   credit_history        1000 non-null   object
 3   purpose               1000 non-null   object
 4   amount                1000 non-null   int64
 5   savings_balance       1000 non-null   object
 6   employment_duration   1000 non-null   object
 7   percent_of_income     1000 non-null   int64
 8   years_at_residence    1000 non-null   int64
 9   age                   1000 non-null   int64
 10  other_credit          1000 non-null   object
 11  housing               1000 non-null   object
 12  existing_loans_count  1000 non-null   int64
 13  job                   1000 non-null   object
 14  dependents            1000 non-null   int64
 15  phone                 1000 non-null   object
 16  default               1000 non-null   object
dtypes: int64(7), object(10) (Table 1)
```

Nulls and nans were checked. Empty cells were treated as nans. The data types of features were noted.

Cardinality of non-numeric attributes implied categorical values.

```
checking_balance          4
months_loan_duration     33
credit_history            5
purpose                   6
amount                  921
savings_balance           5
employment_duration       5
percent_of_income         4
years_at_residence        4
age                      53
other_credit              3
housing                   3
existing_loans_count      4
job                       4
dependents                2
phone                     2
default                   2
Cardinalities (Table 2)
```

Categorical features

Upon inspection credit history, employment duration and phone were found to be suitable as ordinal features because of the clear hierarchy as follows:

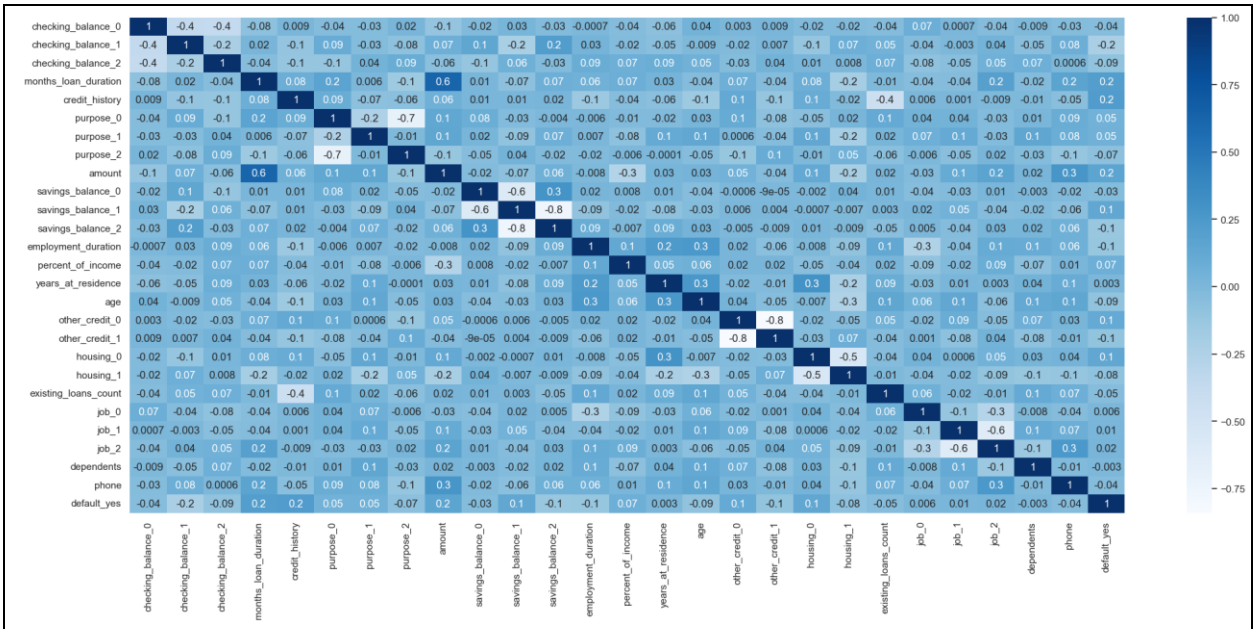
```
['critical', 'poor', 'good', 'very good', 'perfect'],

['unemployed', '< 1 year', '1 - 4 years', '4 - 7 years', '> 7 years'],

['no', 'yes']
```

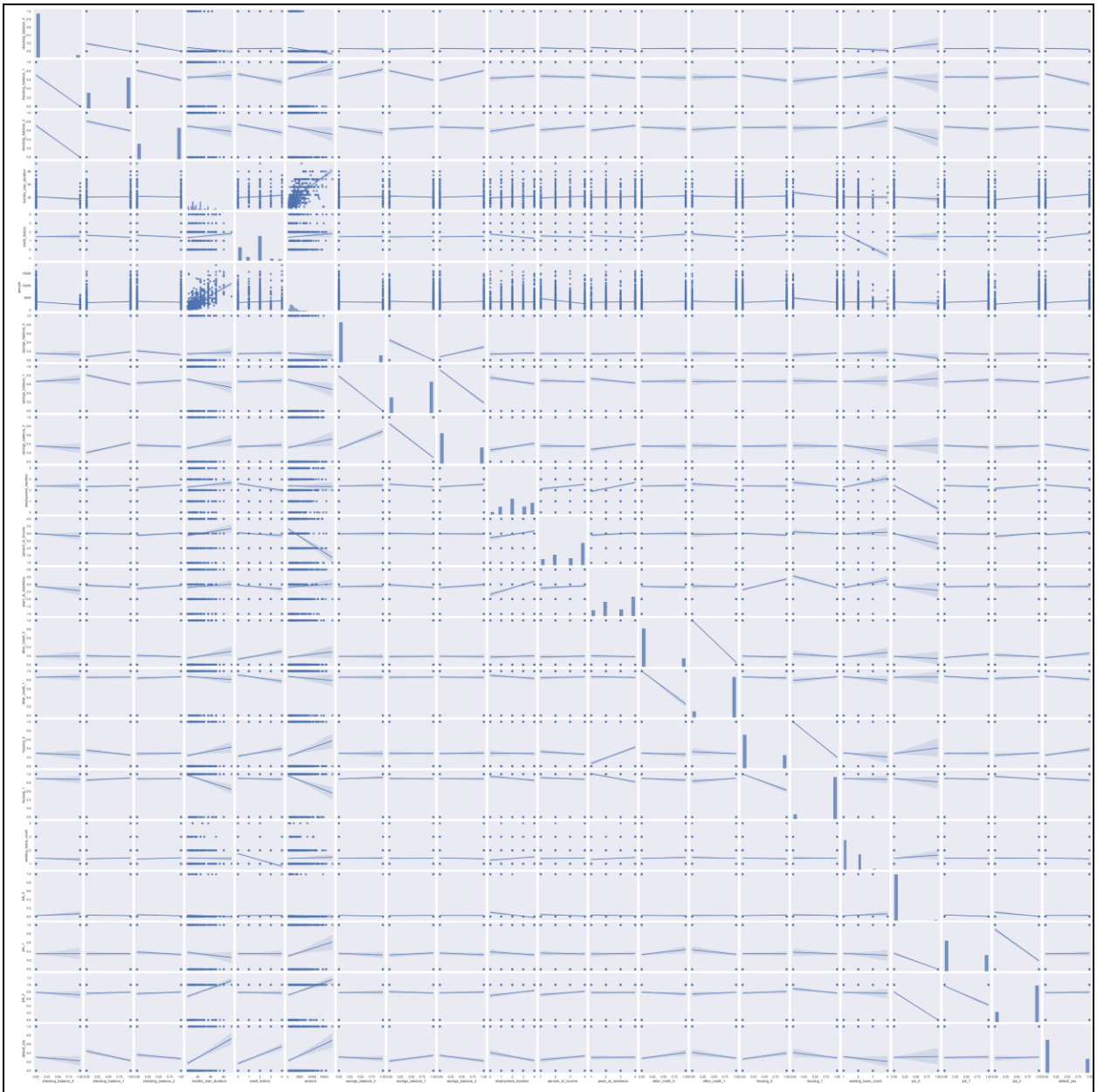
Coverted the above using Ordinal Encoder.[i] The remaining categorical features were binary encoded to form numerical attributes. One Hot Encoding would have resulted in an excess of features, but binary encoder captures the same info in fewer features based on cardinality.[ii] The type of job indicating whether someone is a manager etc was tempting to be used as an ordinal feature, but I chose not to presuppose a job up in the hierarchy to default less often.

Correlation and analysis



Correlation Matrix (Table 3)

Purpose, age, dependents and phone seem to have low correlation. Dropped those and got pairwise plots.



Pairwise Plots (Table 4)

Train Test Split

An 80-20 split of the dataset was obtained for training and testing.[iii]

Model building

Logistic Regression

Decided to try Logistic Regression as the problem at hand is a binary classification problem with a continuous likelihood probability.

```
Confusion Matrix :
 [[117  23]
 [ 26  34]]
Test accuracy =  0.755

Logit Performance (Table 5)
```

The accuracy of 0.755 (0.76 with the dropped features) is not good enough. The recall of 33 out of 54 is not too bad for a loan default prediction system, however, the low precision of 33 out of 60 could hurt genuine loan seekers.

LDA

```
precision    recall  f1-score   support

       False       0.80      0.89      0.84       560
        True       0.64      0.47      0.54       240

    accuracy                           0.76       800
   macro avg       0.72      0.68      0.69       800
weighted avg       0.75      0.76      0.75       800

LDA Performance (Table 6)
```

QDA

```
precision    recall  f1-score   support

       False       0.80      0.89      0.84       560
        True       0.64      0.47      0.54       240

    accuracy                           0.76       800
   macro avg       0.72      0.68      0.69       800
weighted avg       0.75      0.76      0.75       800

QDA Performance (Table 7)
```

KNN

```
precision    recall  f1-score   support

       False       0.73      0.97      0.83       560
        True       0.70      0.15      0.24       240

    accuracy                           0.73       800
   macro avg       0.71      0.56      0.54       800
weighted avg       0.72      0.72      0.65       800

KNN Performance (Table 8)
```

Gradient Boost

```
The mean squared error (MSE) on test set: 0.1635
R2 score on training data:  0.5308242012444484
R2 score on test data:  0.2214491887687865
The mean squared error (MSE) on test set: 0.2115
R2 score on training data:  0.9901683726512466
R2 score on test data:  -0.007335923566492042

Performance for 2 different max depths (Table 9)
```

Random Forest

```
The mean squared error (MSE) on test set: 0.1645
R2 score on training data:  0.3147716412639031
R2 score on test data:  0.21650081627948958
The mean squared error (MSE) on test set: 0.1602
R2 score on training data:  0.8054214865004085
R2 score on test data:  0.23736465287634823

Performance for 2 different max depths (Table 10)
```

Catboost

```
precision    recall  f1-score   support

          no       0.81      0.84      0.82       140
         yes       0.59      0.55      0.57        60

    accuracy                           0.75       200
   macro avg       0.70      0.69      0.70       200
weighted avg       0.75      0.75      0.75       200

bestTest = 0.7781954887
bestIteration = 75
Performance after grid search (Table 10)
```

Conclusion

Following are the key outcomes and observations from the exercise described above.

1. Low correlation on some features, dropping gives about the same performance.
2. Logit gives the best performance at 0.76 accuracy. CatBoost gives about the same performance. The only advantage being that we do not have to convert categorical attributes to numeric in the latter case.
3. Gradient Boosting and Random Forest perform poorly too with our fitting as we increase the depth.
4. One possible area for improvement is in the encoding of categorical features.
    a. `Credit History` - We are not sure how the difference between *critical* and *poor* is vs *very good* and *perfect*.

b. `Employment Duration` - Can we bucket better than the current buckets?
c. `Checking and Savings Balance` - These should ideally have a high predictive power when used as a numeric ordinal feature. However, I've made them binary encoded because of unknown values. If we had more training data, we could try imputing the unknowns.

5. A relatively low LDA f1-score on training data suggests underfitting. Given this, we could perhaps try Target Encoder for a few of the categorical features.
6. We could also make cross-validation based improvements.

---

[i] https://medium.com/@brandon93.w/converting-categorical-data-into-numerical-form-a-practical-guide-for-data-science-99fdf42d0e10

[ii] https://towardsdatascience.com/beyond-one-hot-17-ways-of-transforming-categorical-features-into-numeric-features-57f54f199ea4

[iii] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html